

Computer Organization And Architecture

How and why a computer works

It's difficult to use machine language

so complexity starts to appear

Whenever a it becomes tedious and difficult to use, an **abstraction** is made

Each *layer* of this abstraction has it's own *complexity*

And this course will be going through most of these layers to more easily understand it

Languages

Fundamentally, what a person wants to *do* and what a computer *can* do have a very large gap

Languages, levels, and Virtual Machines

Let's say that we have *two* languages

- **L1** , a language that's easier for a human
- **L0** , the language that the machine uses

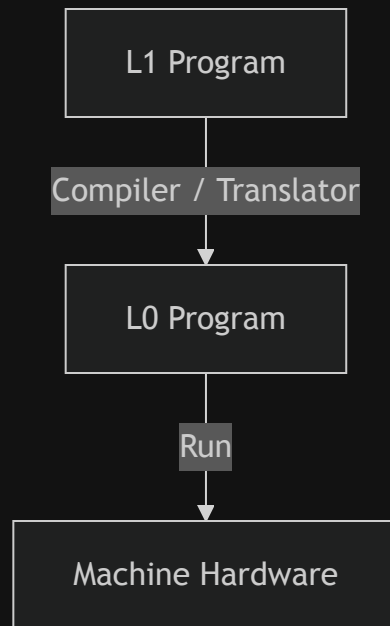
A machine can **only** run programs in the **L0** language

To bridge that gap, there are *two* main methods

Translation

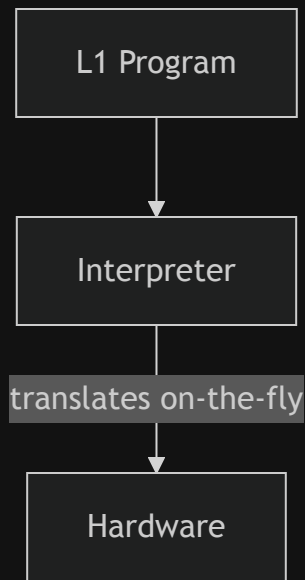
1. Take the L1 program,
2. take each instruction,
3. replace it with one or more L0 instructions that are *equivalent*

In the end, you have a fully L0 program that the machine can run



Interpretation

1. Write an `L0` program that takes in `L1` as input
2. That `L0` program then translates each input as equivalent `L0` code
3. Then that `L0` code is run



Translation vs Interpretation

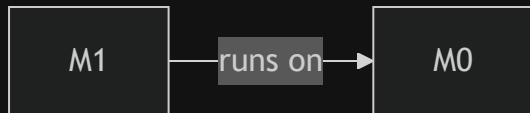
Virtual Machines

One way of thinking about these different layers is through *virtual machines*

Where each *hypothetical machine* runs a *specific language*

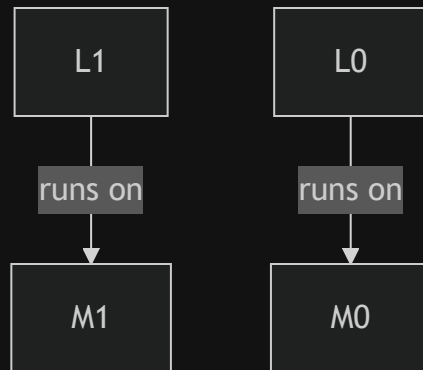
Where **M0** would be a virtual machine that has a **physical** counterpart

And a way to think about *translation* or *interpretation* is that we allow **m1** to run on **m0**



So if **L0** is too difficult, we can make **L1**

These languages (**L0** and **L1**) must not be too different



Usually this means that **L1** is only *slightly* more convenient than **L0**

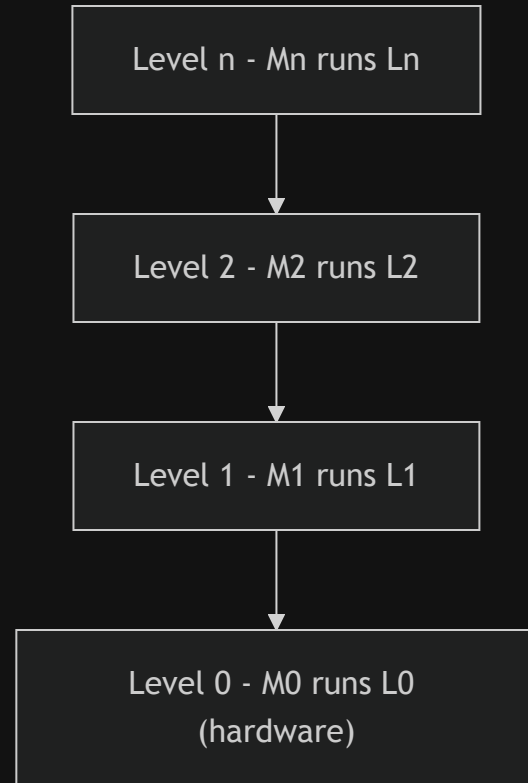
Multilevel machine

A computer with n levels can be regarded as n different virtual machines

Where programs in L_n are

- either *interpreted* by a program running on a *lower machine*, or
- are *translated* to the machine language of a *lower machine*

Note that *level* and *virtual machine* are interchangeable but virtual machine does have a other meanings



Contemporary multi level machines

Contemporary multilevel machines

Most modern computers have two or more levels

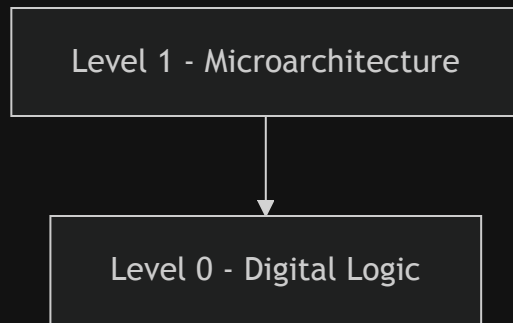
We'll be going through each level

Microarchitecture Level

In this level, we have collections of *registers* and a circuit called the **Arithmetic Logic Unit** (ALU)

The ALU and registers form a *data path*

- *select* one or two registers
- ALU *operates* on them
- *store* result back

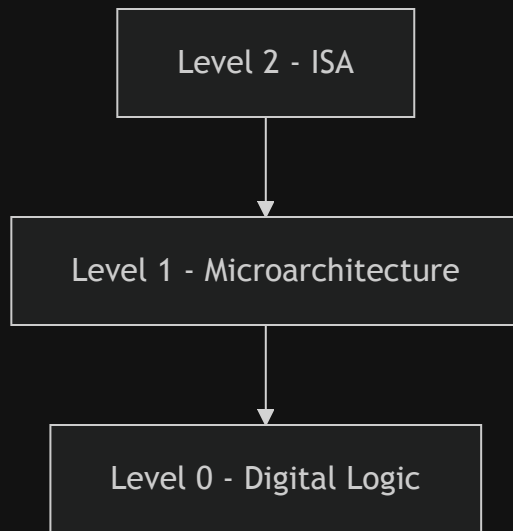


Instruction Set Architecture Level

Every computer manufacturer publishes a manual for each of the computers it sells titled something like "*Machine Language Reference Manual*"

risc-v manual

Level	Example: <code>ADD R1, R2</code>
ISA	instruction fetched & decoded
Microarch	registers selected -> ALU adds
Digital Logic	transistors propagate -> result on bus

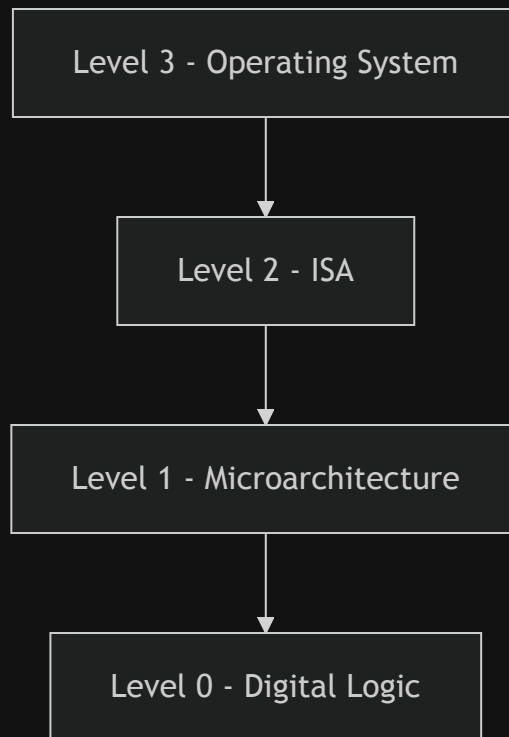


Operating System Level

Adds a set of instructions for

- memory organization
- concurrency and parallelism
- and other features

Note that many operating systems **also** operate *within* the ISA level too



Assembly Language Level

There is a break between levels 3 and 4

The lowest 3 levels are **not** designed to be used by a regular programmer

Level 4 and above is where *applications* start existing

Another difference is that levels 1-3 are *numeric* but assembly starts using *symbols*

At its core assembly is simple a **symbolic** representation of one of the lower levels

And it's then converted into a numeric with an **assembler**

Problem-Oriented Language Level

High-level languages (C, Java, Python, etc.)

A compiler or interpreter translates to lower levels

Provides abstractions like

- variables, functions, objects
- type systems and memory management
- far removed from hardware details

Summary

- Computers are designed as a **series of levels**, each built on its predecessor
- Each level is a distinct **abstraction** suppressing irrelevant detail to reduce complexity

Architecture vs Implementation

- Architecture: visible to the user of that level (data types, operations, features)
- Implementation: how those features are realized (under the hood)

Computer architecture is the study of designing the visible parts of a computer system